

VisionEngine

Бачыце інтэрфейс як карыстальнік — камп'ютарны зрок разам з LLM-зоркам для аналізу і навігацыі.

Зводка

VisionEngine — гэта дэкапліраваны Go-інструментарый, які спалучае класічны камп'ютарны зрок з LLM-зоркам для аналізу карыстальніцкіх інтэрфейсаў, выяўлення элементаў UI і візуальных праблем, а таксама пабудовы графаў навігацыі па пераходах паміж экранамі праграмы — з магчымасцю падлучэння некалькіх правайдэраў зроковых сістэм і OpenCV, які даступны толькі праз будаўнічы тэг.

Кароткае апісанне

Перавыкарыстывальны Go-мадуль для аналізу інтэрфейсаў і пабудовы графаў навігацыі. Ён прапануе ўзровень аналізатара (элементы UI, адрозненні паміж экранамі, візуальныя праблемы), граф навігацыі з пошукам шляху праз BFS і экспартам у фарматах DOT/JSON/Mermaid, а таксама адаптары LLM-зоркі для GPT-4o, Claude, Gemini, Qwen-VL і іншых.

Падрабязнае апісанне

Большасць аўтаматызаваных тэстаў інтэрфейсаў фактычна сляпыя. Яны звяртаюцца да дрэваў даступнасці і селектараў DOM — гэта машыннае ўяўленне пра інтэрфейс — і прапускаюць усё, што сапраўды бачыць чалавек: ці адлюстравалася кнопка, ці не зламалася разметка, ці той самы экран, на які трапіла сістэма. VisionEngine ліквідуе гэтую прабелю, надаючы аўтаматызацыі сапраўднае ўспрыманне — здольнасць глядзець на інтэрфейс і разважаць пра яго так, як гэта рабіў бы чалавек. Ён арганізаваны ў чатыры ўзаемадзейныя ўзроўні, якія паступова пераходзяць ад сырых пікселяў да разумення праграмы ў цэлым. **Аналізатар** вызначае стабільны кантракт — інтэрфейсы (Analyzer, VideoProcessor) і тыпы даных (UIElement, ScreenAnalysis,

ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) з рэферэнснай рэалізацыяй StubAnalyzer — каб карыстальнікі маглі выяўляць элементы, параўноўваць экраны і выяўляць візуальныя праблемы ў межах кантракту, які не зменіцца пад іх нагамі. **Навігацыйны граф** падыходзіць да асобнага экрана на ўзровень усёй праграмы, мадэлюючы яе як накіраваны граф пераходаў паміж экранамі з пошукам шляху праз BFS і трыма фарматамі экспарту (DOT, JSON, Mermaid), каб аўтаматызацыя магла не толькі бачыць экран, але і планаваць шлях да любога іншага — і ў камплект уваходзяць тэставыя наборы для нагрузкі, аўтаматызацыі, інтэграцыі і бяспекі, якія пацвярджаюць яго працаздольнасць. **LLM-зорка** дадае сучаснае мультымадальнае разважанне: інтэрфейс VisionProvider з адаптарамі для OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica і Ollama, аб'яднанымі праз FallbackChain, каб правайдэр, які не спрацоўвае, абмежаваны па частаце запытаў ці слабы, паступова дэградаваў да наступнага, не зводзячы ўвесь працэс на нішто. **Канфігурацыйны ўзровень** апрацоўвае загрузку і праверку зменных асяроддзя, прычым усе паведамленні аб памылках для карыстальніка праходзяць праз i18n.Translator.

Рашэнне, якое робіць усё гэта сапраўды прыдатным для выкарыстання, заключаецца ў тым, што цяжкая натыйная залежнасць з'яўляецца апцыянальнай. Біндынгі OpenCV даступныя толькі праз будаўнічы тэг -tags vision, а ў стандартнай зборцы выкарыстоўваюцца заглушкі — такім чынам, увесь мадуль кампілюецца, тэстуецца і працуе на любой платформе Go 1.25+ без неабходнасці ўсталёўваць OpenCV. Натыйны стэк падключаецца толькі тады, калі карыстальнік свядома вырашае яго выкарыстоўваць. Гэта дазваляе VisionEngine лёгка інтэгравацца ў звычайны CI-ранер без спецыяльнага вобраза. Цалкам дэкапліраваны ў адпаведнасці з канстытуцыяй (CONST-051(B)), ён уключаецца карыстальнікамі — у прыватнасці, HelixQA — як падмадуль з аднолькавай кодовай базай, надаючы UI-тэставанню на аснове фактычных даных сапраўдныя вочы.

Чаму мы яго стварылі

Аўтаматызацыя тэставання інтэрфейсаў, якая абапіраецца выключна на дрэвы даступнасці ці селектары, не бачыць таго, што бачыць карыстальнік. VisionEngine дадае сапраўднае візуальнае разуменне — выяўленне элементаў, параўнанне экранаў і разумовы аналіз на аснове LLM, а таксама навігацыйную карту экранаў дадатку, каб аўтаматызацыя магла не толькі ўспрымаць інтэрфейс, але і перамяшчацца па ім.

Чаму гэта змяняе правілы гульні

Гэта аб'ядноўвае два звычайна несумяшчальныя падыходы — хуткую, дэтэрмінісцкую класічную камп'ютарную зрэнку і гнуткую, семантычную зрэнку на аснове LLM — пад адзіным інтэрфейсам з ланцужком запасных варыянтаў. Такім чынам карыстальнік атрымлівае дакладнасць аднаго і разумовасць другога, не выбіраючы паміж імі. А дзякуючы таму, што OpenCV застаецца строга апцыянальным, знікае звычайны падатак за гэтую магутнасць: любы праект на Go можа атрымаць сапраўднае ўспрыманне інтэрфейсу, не цягнучы за сабой натыўны візуальны тулчэйн у свой білд.

Што новага

- **Двайное ўспрыманне:** класічная камп'ютарная зрэнка (OpenCV/GoCV) плюс мультыправайдарская зрэнка на аснове LLM з ланцужком запасных варыянтаў.
- **Навігацыйны граф** з пошукам шляху праз BFS і экспартам у фарматах DOT/JSON/Mermaid.
- **Адключэнне OpenCV праз білд-тэгі**, каб модуль заставаўся збудаваным і правяраным без натыўных залежнасцей.
- **Поўнасю развязаны, інтэрнацыяналізаваны субмодуль з адзінай кодавай базай** (выкарыстоўваецца ў HelixQA).

Праблемы і рашэнні

- **Праблема цяжкіх натыўных залежнасцей:** вырашана праз гейтынг -tags vision і стандартныя заглушкі, каб CI і хосты без OpenCV усё роўна збіраліся і тэставаліся.
- **Нестабільнасць правайдараў візуальнага аналізу:** вырашана праз інтэрфейс VisionProvider і кампазітар FallbackChain.
- **Мадэляванне складаных патокаў дадатка:** вырашана праз накіраваны навігацыйны граф з пошукам шляху праз BFS і экспартам у некалькіх фарматах.
- **Залежнасці:** вырашана праз прынцып CONST-051(B) і слой інтэрнацыяналізацыі.

Тэхналагічны стэк (навошта і як)

- **Go (1.25+)** — ядро модуля і ўсе чатыры ўзроўні.
- **GoCV / OpenCV** — класічная камп'ютарная зрэнка, якая падключаецца праз білд-тэгі.

- **Правайдары зрэнкі на аснове LLM (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — мультымадальны аналіз інтэрфейсу праз адаптары.
- **Алгарытмы графаў (BFS)** — пошук навігацыйных шляхоў.
- **Экспарцёры DOT / JSON / Mermaid** — візуалізацыя навігацыйнага графа.
- **Транслятар інтэрнацыяналізацыі** — развязаныя радкі для карыстальніцкага інтэрфейсу.